

CSA0315 – DATA STRUCTURES
CO4- AT1- COMPARITIVE ANALYSIS.

Faculty:

Dr. T. Rajesh Kumar

By:

Harshith G (192525394)

1) A system needs to store large data and perform frequent search operations. For this, compare Hashing and Sorting-based search (Binary Search)

ANS:

Factor	Hashing	Sorting + Binary Search
a) Time complexity	Search: $O(1)$ average, $O(n)$ worst case	Search: $O(\log n)$
Initial preparation	$O(n)$ to build hash table	$O(n \log n)$ to sort
b) Space usage	Usually $O(n)$ extra space for hash table	Binary search itself uses $O(1)$ extra space if iterative
c) Update operations	Insertion and deletion: $O(1)$ average	Insertion/deletion can be $O(n)$ because elements may need shifting/reordering
d) Real-world applicability	Very useful for databases, caches, dictionaries, symbol tables, and fast lookup systems	Useful when data is already sorted and when ordered/range queries are required

Conclusion

For a dynamic system with frequent insertions, deletions, and searches, Hashing is generally better because it provides average $O(1)$ search, insertion, and deletion.

However, Binary Search is preferable when:

- Data must remain sorted.
- Range queries are important.
- Ordered traversal is required.

2) Compare Quick Sort and Merge Sort Explain why Quick Sort is often preferred in real applications despite its worst-case complexity.

ANS:

Factor	Quick Sort	Merge Sort
a) Best case	$O(n \log n)$	$O(n \log n)$
Average case	$O(n \log n)$	$O(n \log n)$
Worst case	$O(n^2)$	$O(n \log n)$
b) Stability	Generally not stable	Stable
c) Memory usage	Usually $O(\log n)$ auxiliary space for recursion	$O(n)$ auxiliary space for arrays
d) Practical performance	Often very fast due to good cache performance and low overhead	Predictable performance and very suitable for large/external datasets

Why is Quick Sort often preferred despite $O(n^2)$ worst case?

Quick Sort is frequently preferred in real applications because its average-case performance is $O(n \log n)$ and it has several practical advantages:

1. **Low memory requirement** – It can operate largely in-place, unlike Merge Sort which normally requires $O(n)$ additional memory.
2. **Good cache performance** – Quick Sort works mostly within the same array, making it efficient with modern CPU cache memory.
3. **Low constant factors** – Its operations can be very fast in practice.
4. **In-place sorting** – It does not normally require a separate array for merging.
5. **Worst case can be reduced** – Choosing a good pivot, such as a random pivot or median-of-three, greatly reduces the chance of encountering the $O(n^2)$ case.